

Data Structure and Algorithms



Objectives



- Algorithms
- Big O Notations

Algorithms

- A set of steps or instructions for completing a task

Example:

A recipe

A todo-list

driving algorithms



Classic river-crossing puzzle

- ቀበሮ፣ ፍየል እና ጎመን puzzle environment



3 Priests and 3 devils Puzzle

- There are 3 devils and 3 Priests. They all must cross a river in a boat. Boat can only carry two people at a time.
- As long as there are equal number of devils and priests, then devils will not eat Priest. If the number of devils are greater than the number of priests on the same side of the river then devils will eat the priests. So how can we make all the 6 peoples to arrive to the other side safely?





Algorithmic thinking

- **Algorithmic thinking** is a **problem-solving approach** that involves breaking down complex tasks into smaller, more **manageable** steps or procedures.
- It is the process of designing and implementing algorithms to solve specific problems efficiently and effectively.
- Key **aspects** of Algorithmic thinking:
 - Problem Decomposition
 - Pattern Recognition
 - Abstraction
 - Algorithm Design
 - Efficiency Consideration
 - Testing and Debugging
 - Algorithm Analysis
 - Iterative Improvement



Cont'd

- Problem Decomposition

Breaking down a problem into smaller, more manageable subproblems or tasks.

- Pattern Recognition

Identifying common patterns or structures within the problem that can be exploited to develop a solution.

- Abstraction

Focusing on the essential details while ignoring unnecessary or irrelevant information to create a simplified representation of the problem.

- Efficiency Consideration

Striving to design algorithms that are efficient in terms of time complexity (execution time) and space complexity (memory usage).



counting-out game [Josephus problem]

1. All of you Stand Up and Think of Number 1:

Each person initially stands up and thinks of the number 1.

2. Pair Off and Add Numbers:

Each person pairs off with someone else standing nearby. They then add their number to the number of their partner.

3. Sit Down:

After adding numbers, one person from each pair sits down. The decision of who sits down can be arbitrary or based on certain rules.

4. Repeat:

If there are still people standing, the process repeats. Those who are still standing pair off with someone else standing nearby, add their numbers, and one person from each pair sits down.



Let's play a game

- I'm going to play a guessing game with two of you.
- Think of any number between 1 and 10.
- [Students are instructed to close their eyes or cover their faces for a moment so they don't see the next part.]



Alright, now keep that number in mind.
Let's proceed.

First Number: 3

Second Number: 10

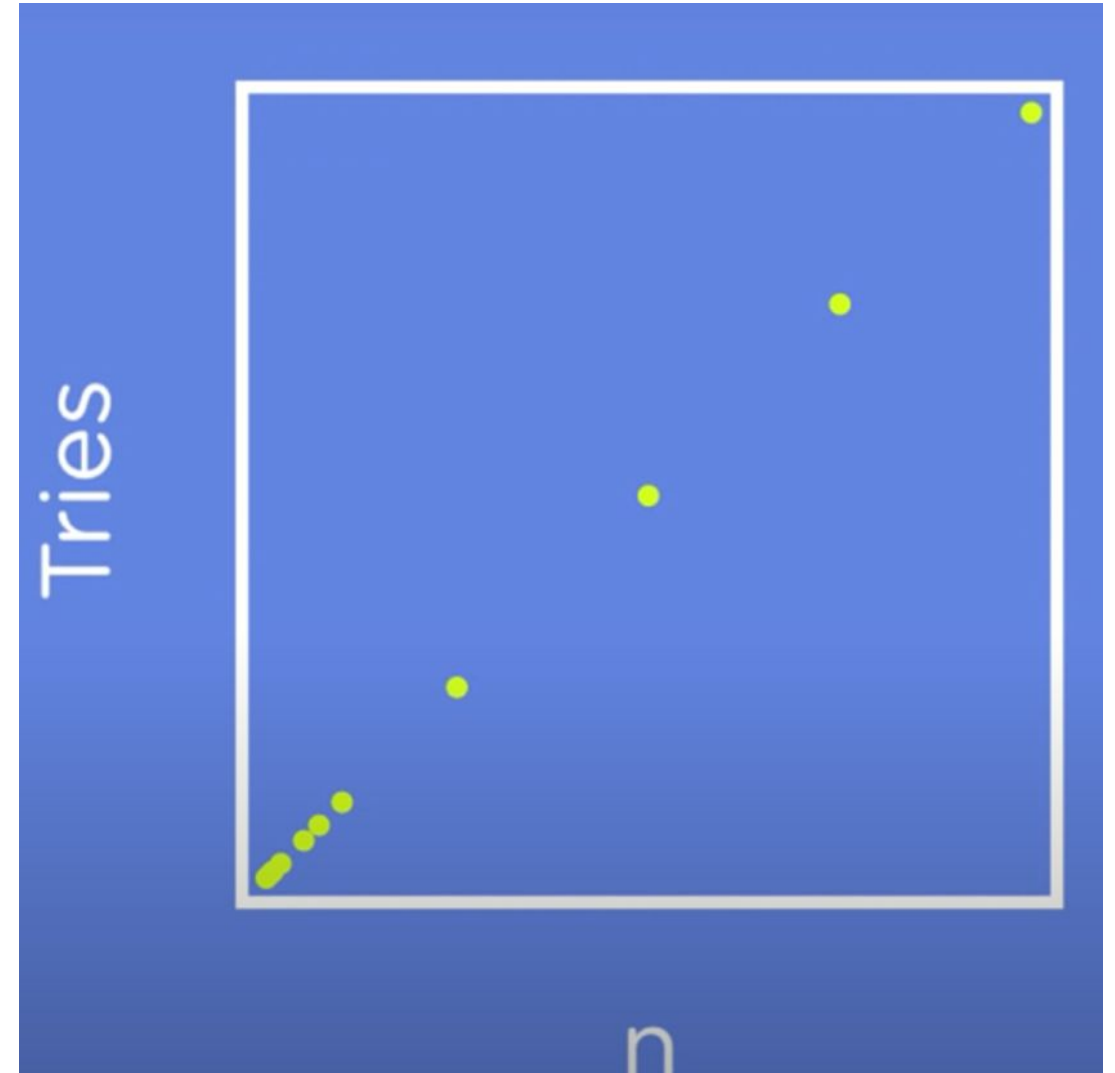


Terminologies during our games

- Best case scenario
- Worst case scenarios
- Number of attempts/tries/times or run time
- Linear search
- Binary search
- “random” search

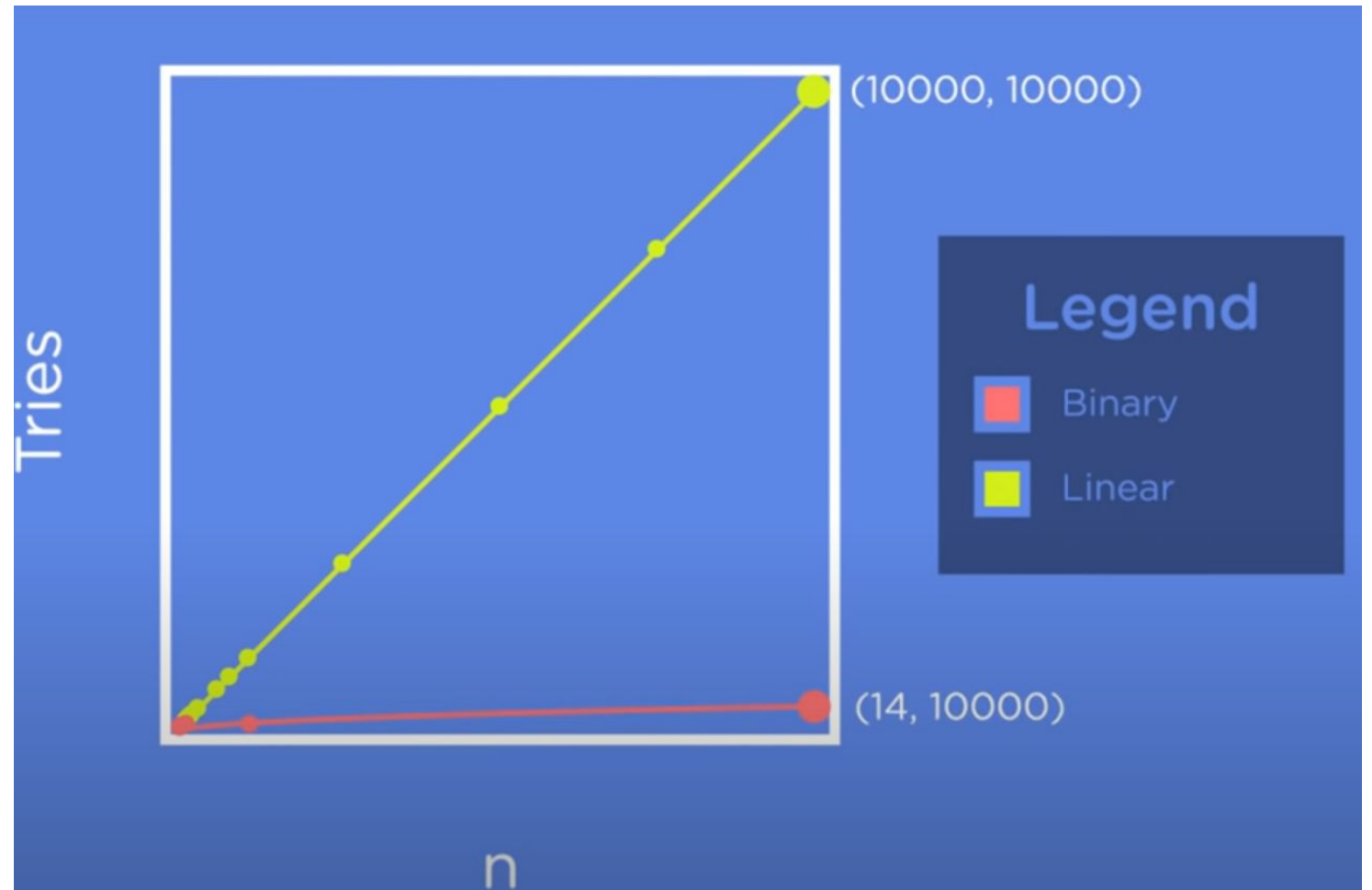
Linear Search during the game

- Tries vs N (max value, worst case)



Binary Search during our games

Order of Growth





- (In Computers) A set of steps a program takes to finish a task



- a finite sequence of instructions, each of which has a clear meaning and can be performed with a finite amount of effort in a finite length of time
- Engineers use pseudocodes to first draft and describe algorithms
“somewhere between formatted English and computer program code”



Ethiopian Binary Math

- A finite sequence of rigorous instructions, typically used to solve a class of specific problems or to perform a computation.
- a specific procedure for solving a well-defined computational problem

- Example:

Ethiopian binary math:

a Shaman in Ethiopian villages:1941

An even number of stones makes
the hole evil.

The shaman's multiplication of 7 x 34:

row #1		row #2	the calculation	
34 stones	(evil)	7	evil	0
17	(good)	14	good	14
8	(evil)	28	evil	0
4	(evil)	56	evil	0
2	(evil)	112	evil	0
1	(good)	224	good	224

$$238 = 7 \times 34$$



```
int ethiopianMultiplication(int n1, int n2) {  
  
    int result = 0;  
  
    while (n1 != 1) {  
        if (n1 % 2 != 0) {  
            result += n2;  
        }  
        n1 = n1 / 2;  
        n2 = n2 * 2;  
    }  
  
    result += n2; // Adding the final multiplication when num1 is 1  
  
    return result;  
}
```



- **Algorithms** provide step by step instructions on solving specific problems. They help you solve problems using efficient, standard, and reusable steps
- **Characteristics of an Algorithm:**
 - Input/Output:** defined problem statement, input and output
 - Determinism:** For a given set of inputs, the algorithm should always produce the same output.
 - Definiteness:** Each step of an algorithm must be precisely defined and unambiguous to avoid confusion or misinterpretation.
 - Finiteness:** it doesn't run indefinitely clearly
 - Effectiveness**
 - Generality:** applicable to a wide range of inputs
 - Feasibility**
 - Correctness:** correct output for all valid inputs



Fundamental questions about algorithms

- What is it supposed to do? [**specification**]
- Does it really do what it is supposed to do? [**verification**]
- How efficiently does it do it? [performance analysis | **efficiency**]



- It takes a set of input(s) and produces the desired output.

For example:

An algorithm to add two numbers:

1. Take two number inputs
2. Add numbers using the + operator
3. Display the result



Examples of algorithms

- Searching algorithms
- Sorting algorithms



Performance of an algorithm

- An algorithm is said to be efficient and fast, if it takes **less time** to execute and consumes **less memory** space.

The performance of an algorithm is measured on the basis of following properties :

- **Time** Complexity
- **Space** Complexity



- **Time complexity** of an algorithm quantifies the amount of **time taken by an algorithm to run** as a function of the length of the input.
- Similarly, Space complexity of an algorithm quantifies the amount of space or memory taken by an algorithm to run as a function of the length of the input.



Space complexity

- Its the **amount of memory space** required by the algorithm, during the course of its execution.
- An algorithm generally requires space for following components :

Instruction Space: Its the space required to store the **executable** version of the program. This space is fixed but varies depending upon the number of lines of code in the program.

Data Space: Its the space required to store all the constants and variables(including temporary **variables**) value.

Environment Space: Its the space required to store the environment information needed to resume the suspended function.



restatement

1. Instruction Space

It's the amount of memory used to save the compiled version of instructions.

2. Environmental Stack

Sometimes an algorithm(function) may be called inside another algorithm(function). In such a situation, the current variables are pushed onto the system stack, where they wait for further execution and then the call to the inside algorithm(function) is made.

For example, If a function **A()** calls function **B()** inside it, then all the variables of the function **A()** will get stored on the system stack temporarily, while the function **B()** is called and executed inside the function **A()**.

3. Data Space

Amount of space used by the variables and constants.



Calculating the Space Complexity

- For calculating the space complexity, we need to know the value of memory used by different type of datatype variables, which generally varies for different operating systems, but the method for calculating the space complexity remains the same.

Example:

Bool, char : 1 byte
short, int: 2 bytes
float, long: 4 bytes
double: 8 bytes

Example

```
{  
    int z = a + b + c;  
    return(z);  
}
```

In the above expression, variables **a**, **b**, **c** and **z** are all integer types, hence they will take up 4 bytes each, so total memory requirement will be $(4(4) + 4) = 20$ bytes, this additional 4 bytes is for **return value**.

Example

```
// n is the length of array a[]
int sum(int a[], int n)
{
    int x = 0;          // 4 bytes for x
    for(int i = 0; i < n; i++) // 4 bytes for i
    {
        x = x + a[i];
    }
    return(x);
}
```

- $4*n$ bytes of space is required for the array $a[]$ elements.
- 4 bytes each for x , n , i and the return value.
- Hence the total memory requirement will be $(4n + 12)$, which is increasing linearly with the increase in the input value n



Time complexity

- Time Complexity is a way to represent the amount of time required by the program to run till its completion. It's generally a good practice to try to keep the time required minimum, so that our algorithm completes its execution in the minimum time possible
- Time complexity is a measure of the amount of time an algorithm takes to run as a function of the input size.

- The time complexity describes the amount of time necessary to execute an algorithm.
- However, it is not a measure of the actual time taken to run an algorithm but a measure of how the time taken grows with respect to change in size of the input.



Are we using “Time” timers? No

- Hardware dependency

Time measurements can vary based on the hardware the algorithm is executed on.

- External interference

Other processes running on the system can interfere with timer measurements, affecting accuracy.

- Granularity

Timer granularity may be too fine to accurately measure algorithms with lower time complexity. Using seconds and milliseconds might not accurately measure algorithms with lower time complexity

- Algorithm variation

Timer measurements may not reflect the growth rate of an algorithm's running time, but rather absolute time for specific inputs.



Solution? Theoretical analysis

- So then what do we use if not time? The simple answer is **counting operations**. Let's explore some examples of what this means.
- This involves analyzing the algorithm's code and determining the number of basic operations (such as comparisons and assignments) executed as a function of the input size. This gives an understanding of the algorithm's growth rate and thus its time complexity.



Example: Traverse an array: Does x exist in Array A ?

```
for i : 1 to length of A
    if A[i] is equal to x
        return True
return False
```

Each of the operation in computer take approximately constant time.

Let each operation takes c time. The number of lines of code executed is actually depends on the value of x .

During analyses of algorithm, mostly we will consider worst case scenario, i.e., when x is not present in the array A .

In the worst case, the **if** condition will run N times where N is the length of the array A .

So in the worst case, total execution time will be $N*c + c$. $N*c$ for the **if** condition and c for the **return** statement (ignoring some operations like assignment of i).



- As we can see that the total time depends on the length of the array A.

If the length of the array will increase the time of execution will also increase.



Find the square of a number

Method 1:

```
for i=1 to n
  do n = n + n
// when the loop ends n will hold its square
return n
```

Method 2:

```
return n*n
```

Which one do you prefer?

Looping n times or just multiplying the number by itself?

Hope you choose the second one.
But why?



Pseudocode /'soōdō,kōd/

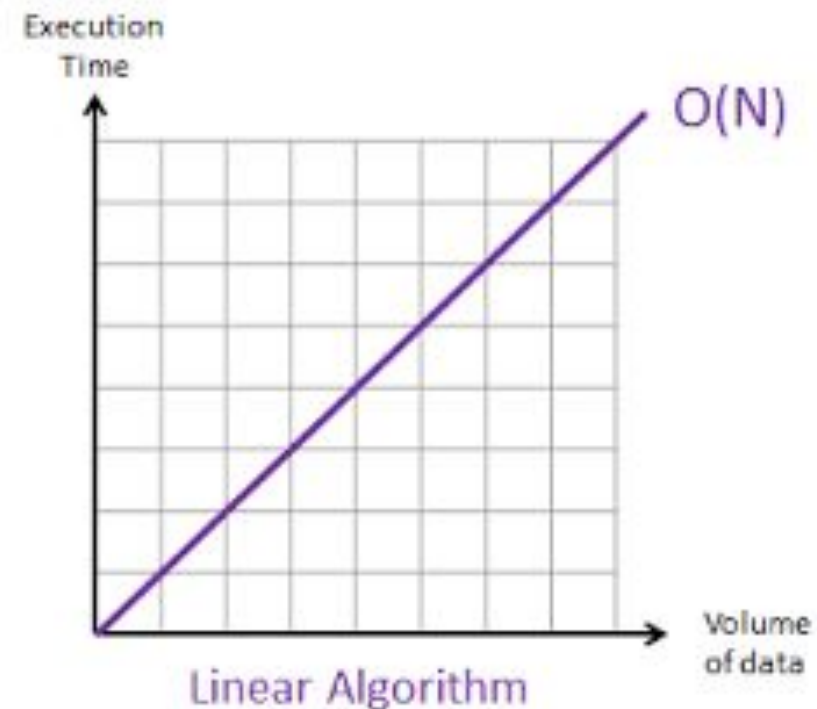
- A structured representation of algorithmic steps.
- Human-readable format, independent of programming syntax.
- Acts as a bridge between high-level algorithmic descriptions and code implementation.
- Utilizes natural language, basic control structures, and algorithm-specific notation.



Linear Search pseudocode

```
LinearSearch(array, target):  
    for each element in array:  
        if element equals target:  
            return index of element  
    return "Not found"
```

- The loop runs N times, where N is the length of the array.
- If an operation has an $O(n)$ (pronounced 'O of N'), what this means is that, as ' n ' increases, so does the amount of operations needed to complete the problem.





Binary search pseudocode

BinarySearch(array, target):

```
left = 0
right = length of array - 1

while left <= right:
    mid = (left + right) / 2
    if array[mid] == target:
        return mid
    else if array[mid] < target:
        left = mid + 1
    else:
        right = mid - 1
return "Not found"
```



Big O

- Theoretical definition of the complexity of an algorithm as a function of the size
- A notation to describe complexity

“O” – order of magnitude of complexity



- Simply put, Big O (a capital letter O, not a zero) Notation is a measure of the complexity of an algorithm, a measurement of how quickly an algorithm's runtime grows relative to its input.
- The letter "O" signifies the growth rate of a function, also called its order.

- Takeaway

Big O Notation is NOT a measure of how quick your algorithm is, rather a measure of how much more work an algorithm will have to perform as the size of its input gets larger.



Order of growth (asymptotic notation, growth rate)

- **Order of growth** is how the time of execution depends on the length of the input. In the Array Traverse example, we can clearly see that the time of execution is linearly depends on the length of the array. Order of growth will help us to compute the running time with ease.
- We use different notation to describe limiting behavior of a function.

Types of asymptotic notations:

O- notation (big oh)

Ω - notation (big mega)

Θ - notation (big Theta)



- When the input size increases, does the algorithm become incredibly slow? Is it able to maintain its fast run time as the input size grows? You can answer these questions thanks to Asymptotic Notation.
- You compare space and time complexity using asymptotic analysis. It compares two algorithms based on changes in their performance as the input size is increased or decreased.



- Asymptotic notation is a mathematical notation used to describe the behavior and performance of algorithms and data structures as their input size approaches infinity.
- It's a way to analyze the efficiency of algorithms in terms of how their running time or space requirements grow relative to the size of the input. In data structures, asymptotic notation is often used to discuss the time complexity of various operations.

Example: Two algorithms to construct long strings in Java

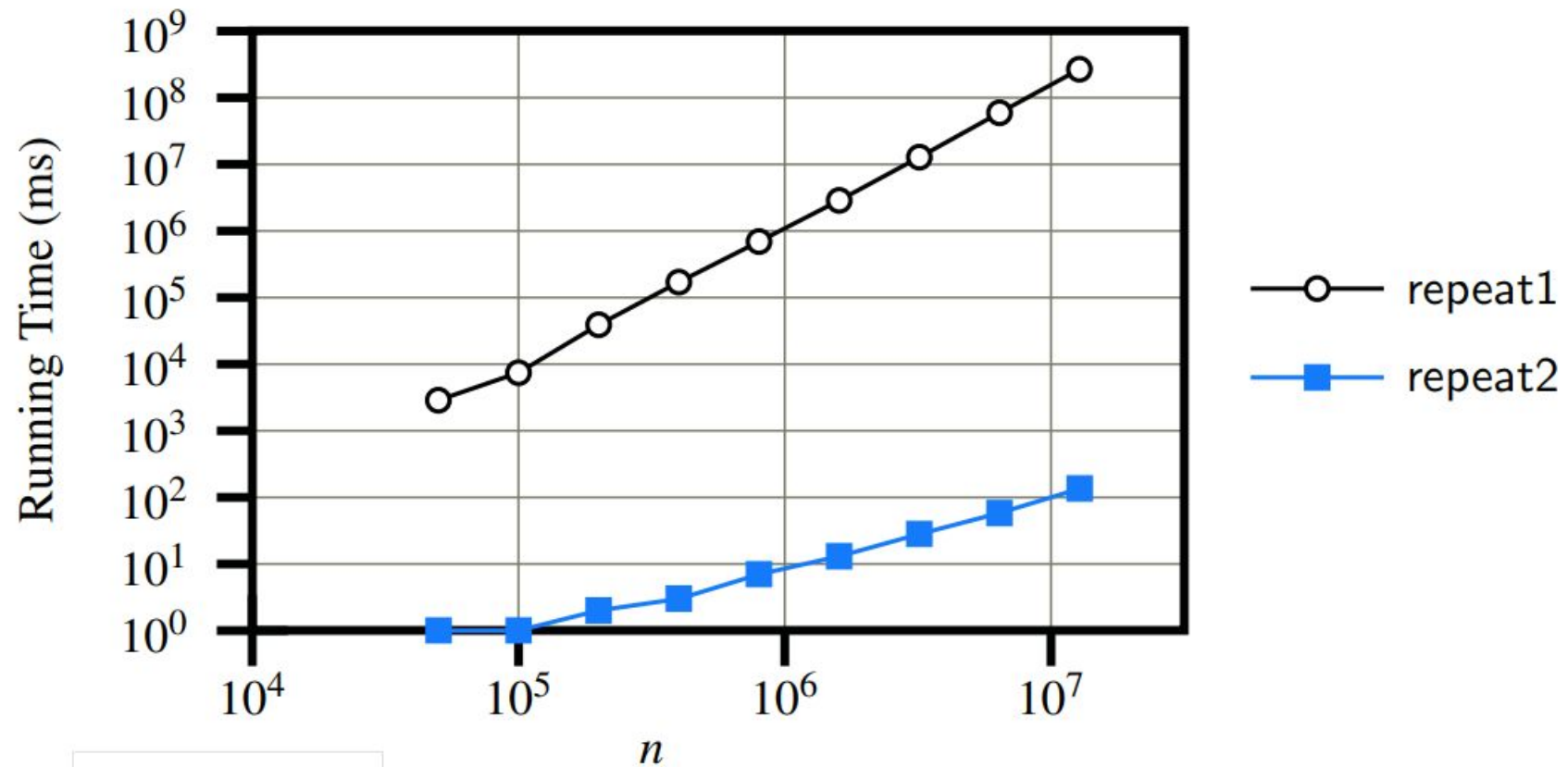
```
String repeat1(char c, int n) {
    String answer = "";
    for(int j = 0; j < n; j++) {
        answer += c;
    }
    return answer;
}

String repeat2(char c, int n) {
    StringBuilder sb = new StringBuilder();
    for(int j = 0; j < n; j++) {
        sb.append(c);
    }
    return sb.toString();
}
```

Results of timing experiment on the methods

n	repeat1 (in ms)	repeat2 (in ms)
50,000	2,884	1
100,000	7,437	1
200,000	39,158	2
400,000	170,173	3
800,000	690,836	7
1,600,000	2,874,968	13
3,200,000	12,809,631	28
6,400,000	59,594,275	58
12,800,000	265,696,421	135

Chart of the results of the timing experiment from Code



Observation

- The most striking outcome of these experiments is how much faster the **repeat2** algorithm is relative to **repeat1**. While **repeat1** is already taking more than 3 days to compose a string of 12.8 million characters, repeat2 is able to do the same in a fraction of a second.
- We also see some interesting trends in how the running times of the algorithms each depend upon the size of **n**. As the value of **n** is doubled, the running time of **repeat1** typically increases more than fourfold, while the running time of **repeat2** approximately doubles.



Asymptotic notations

- Mathematical tools used in algorithm analysis to describe the behavior of functions as their input size approaches infinity.
- Asymptotic analysis studies the behavior of functions as their input size approaches infinity.
- It focuses on understanding the limiting behavior of functions rather than their exact values.
- Asymptotic notations, including Big O notation, are used to express the growth rates of functions in terms of simple, easy-to-understand formulas.

Example: Big O notation



Asymptotic Notations

- **Big O Notation (O):** This represents an upper bound on the growth rate of an algorithm's running time. It describes the **worst-case** scenario. If an algorithm has a time complexity of $O(f(n))$, it means that the running time of the algorithm won't grow faster than a constant multiple of $f(n)$ for sufficiently large input sizes.
- **Omega Notation (Ω):** This represents a lower bound on the growth rate of an algorithm's running time. It describes the **best-case** scenario. If an algorithm has a time complexity of $\Omega(g(n))$, it means that the running time of the algorithm won't grow slower than a constant multiple of $g(n)$ for sufficiently large input sizes.
- **Theta Notation (Θ):** This represents **both upper and lower bounds** on the growth rate of an algorithm's running time. It indicates the **average bound** of an algorithm. It represents the average case of an algorithm's time complexity.



Big O

- It indicates the maximum required by an algorithm for all input values. It represents the worst case of an algorithm's time complexity.
- This notation describes the upper bound of an algorithm's time complexity, or the worst-case scenario.
- For example, if an algorithm has a time complexity of **$O(n)$** , where n is the size of the input data, it means that the algorithm will take at most n operations to complete.



Big O Notation

- Time complexity of an algorithm signifies the total time required by the program to run till its completion.
- The time complexity of algorithms is most commonly expressed using the **big O notation**. It's an asymptotic notation to represent the time complexity.

Example: accessing, Insertion in an Array

- You can access any element using its index with a constant time.

index 0
↑
[10 , 9, 6, 5, 8]

For an array, accessing elements at a specified index has a constant time of **Big O(1)**.



- We can access each element by its index; $\text{array}[0] = 10$ and $\text{array}[4] = 8$. Since the elements are indexed, our computers know exactly where the element is and can go directly to the element. Moreover, an array with 2 elements or 2,000 elements has the same time complexity, **$O(1)$** , for accessing methods.



Constant functions

- We saw a constant function right?

$f(n) = c$, for some fixed constant c , such as $c = 5$, $c = 67$, $c = 2^{10}$

That is, for any argument n , the constant function **$f(n)$** assigns the value **c** . In other words, it does not matter what the value of n is; $f(n)$ will always be equal to the constant value c

$f(n) = c * g(n)$; where $g(n) = 1$

Because we are most interested in integer functions, the most fundamental constant function is $g(n) = 1$, and this is the typical constant function we use in this session.

As simple as it is, the constant function is useful in algorithm analysis because it characterizes the number of steps needed to do a basic operation on a computer, like adding two numbers, assigning a value to a variable, or comparing two numbers



- When the algorithm is not dependent on the size of the input, it is said to have a constant time complexity with order $O(1)$. This means that the run time will always be the same regardless of the size of the input.



Example:

```
public class Main {  
    public static void main(String[] args) {  
        int num1 = 10;  
        int num2 = 10;  
        System.out.println(num1 + num2);  
    }  
}
```

Notice that even though the code above would be considered something around **O(3)**, the number 3 is irrelevant because it doesn't make much of a difference. Also, it doesn't matter how many times we executed this algorithm it will have always a constant time complexity. Therefore, the code above also has **O(1)** of time complexity.



- Inserting or removing from an array can come in three different forms: inserting/removing from the **start**, inserting/removing from the **end**, or inserting/removing from the **middle**.
- In order to add an element to the beginning of an array, we must shift every other element after it to a higher index. For example, If we wanted to add 2 to the beginning of the above so that it would now be at the zeroth index, 10 would now be at the first, 9 would be at the second and so on.

Time taken will be proportional to the size of the list or **Big O(n)**, n being the size of the list.



Big O of operations on Arrays

	Array
Cost of accessing elements	$O(1)$
Insert/Remove from beginning	$O(n)$
Insert/Remove from end	$O(1)$
Insert/Remove from middle	$O(n)$



- Time Complexity is most commonly estimated by counting the number of elementary steps performed by any algorithm to finish execution.
- Like in the previous example, for the first code the loop will run n number of times, so the time complexity will be n at least and as the value of n will increase the time taken will also increase.
- While for the second code, time complexity is constant, because it will never be dependent on the value of n , it will always give the result in 1 step.



Calculating time complexity

- Now the most common metric for calculating time complexity is Big O notation. This removes all constant factors so that the running time can be estimated in relation to N, as N approaches infinity.

In general you can think of it like this :

statement;

- Above we have a single statement. Its Time Complexity will be Constant. The running time of the statement will not change in relation to N.



loop

```
for(i=0; i < N; i++)  
{  
    statement;  
}
```

The time complexity for the above algorithm will be **Linear**.

The running time of the loop is directly proportional to N. When N doubles, so does the running time.

```
void printArray(int[] arr) {  
    for(int n : arr) {  
        System.out.println(n);  
    }  
}
```



- In this example, the input is an array **arr** containing **n** elements. The loop iterates over each element in the array exactly once and prints it. The time complexity of this loop is **$O(n)$** because the number of iterations is directly proportional to the size of the input array.
- No matter how large the array becomes, the loop will iterate through each element one by one. This linear relationship between the input size and the number of iterations is why we say the time complexity is $O(n)$, where n is the size of the input.
- It's important to note that the constant factors or lower-order terms are not considered in big O notation. Whether the loop does a simple operation like printing or a more complex operation, as long as the number of iterations is linearly related to the input size, the time complexity will remain $O(n)$.

Linear search example

```
int linearSearch(int[] arr, int target) {  
    for (int i = 0; i < arr.length; i++) {  
        if (arr[i] == target) {  
            return i; // Element found at index i  
        }  
    }  
    return -1; // Element not found in the array  
}
```



Contd.

- In a linear search, the worst-case scenario occurs when the element being searched for is either the last element in the array or is not present in the array at all. In both cases, the algorithm has to check every element in the array.
- The worst-case time complexity of the linear search algorithm is **$O(n)$** , where n is the number of elements in the array. This means that the number of comparisons the algorithm makes grows linearly with the size of the array.



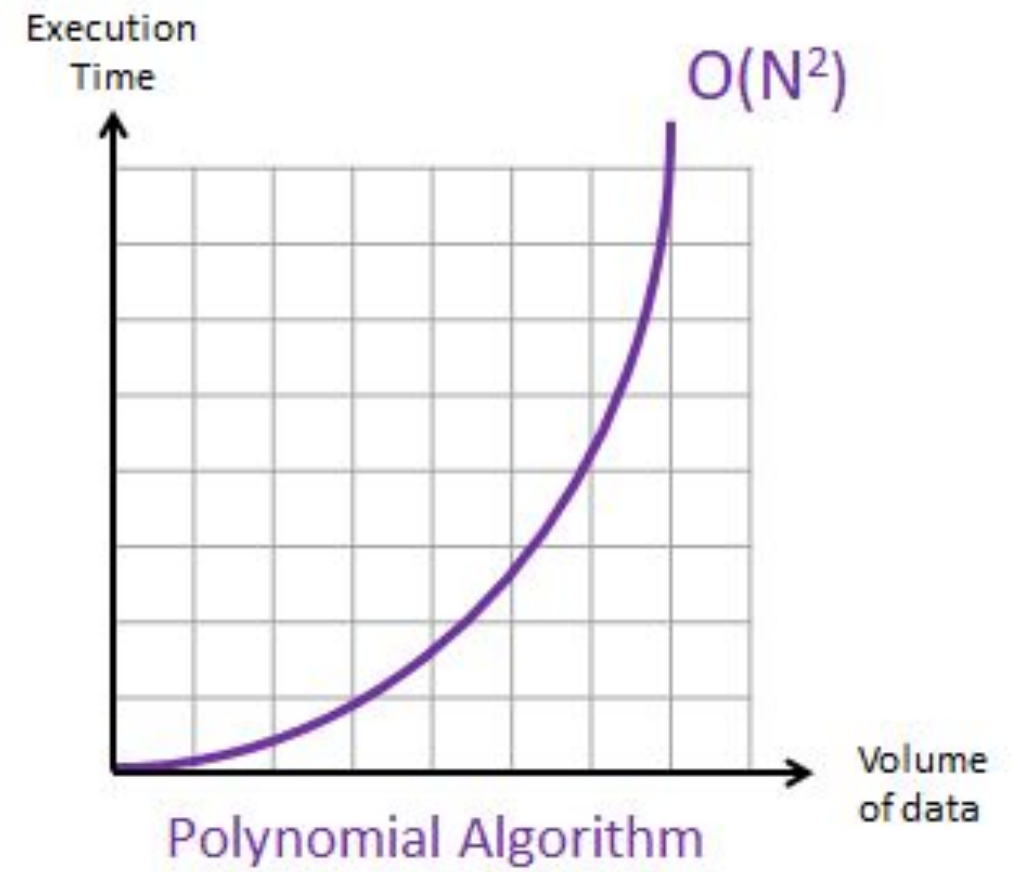
- You saw linear function, right?
 $f(n) = n$



Nested loop

```
for(i=0; i < N; i++)  
{  
    for(j=0; j < N; j++)  
    {  
        statement;  
    }  
}
```

This time, the time complexity for the above code will be **Quadratic**.
The running time of the two loops is proportional to the square of N.
When N doubles, the running time increases by $N * N$.



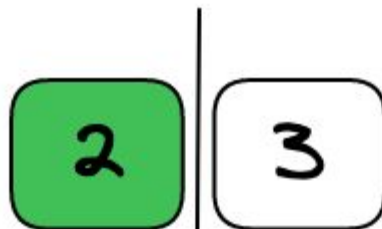
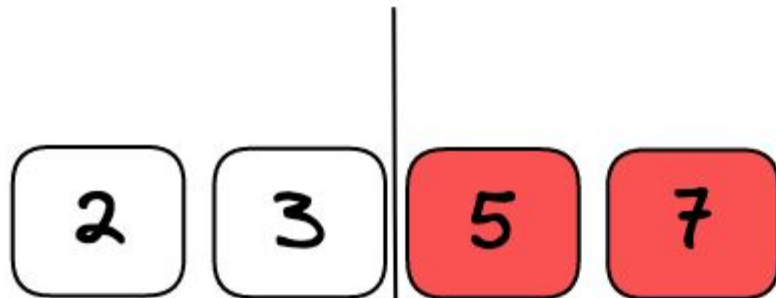
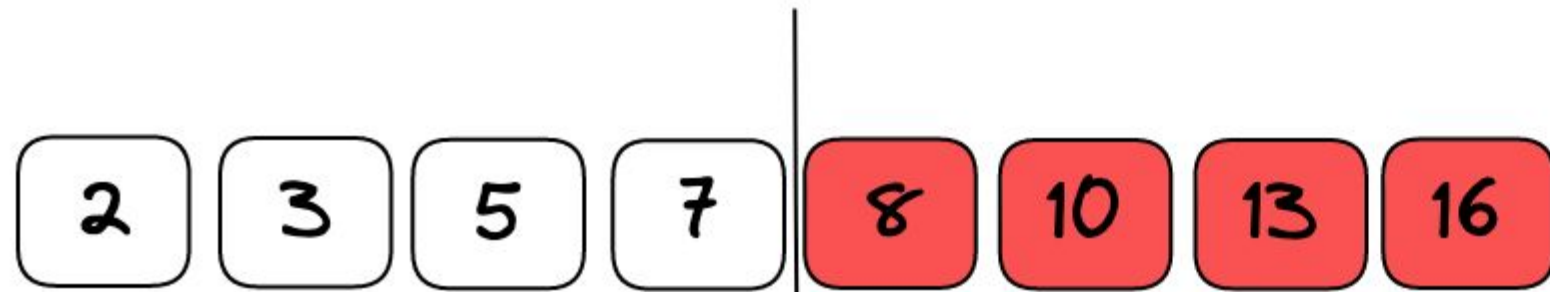


```
while(low <= high)
{
    mid = (low + high) / 2;
    if (target < list[mid])
        high = mid - 1;
    else if (target > list[mid])
        low = mid + 1;
    else break;
}
```

This is an algorithm to break a set of numbers into halves, to search a particular field (we will study this in detail later). Now, this algorithm will have a **Logarithmic Time Complexity**.

The running time of the algorithm is proportional to the number of times N can be divided by 2 (N is high-low here). This is because the algorithm divides the working area in half with each iteration.

number to be found 2



log time complexity

$$2^3 = 8$$

3 elementary operations



```
public void continuallySplit(int n) {  
    while(n > 0) {  
        System.out.println(n);  
        n /= 2;  
    }  
}
```

Time Complexity:

In each iteration of the loop, the value of n is halved.

The number of loop iterations depends on how many times n can be divided by 2 until it becomes less than or equal to 0.

Let's denote the initial value of n as n_0 .

The number of loop iterations required until n becomes 0 can be calculated as the logarithm base 2 of n_0 , rounded down to the nearest integer.

Therefore, the time complexity of this function is $O(\log n)$, where n represents the input size.



When $n = 1$, the loop will run once.

When $n = 2$, the loop will run twice.

When $n = 4$, the loop will run three times.

When $n = 8$, the loop will run four times.

And so on, until $n = 2^k$, the loop will run k times.

In general, the loop will run k times when $n = 2^K$, where K is the integer logarithm base 2 of n .

So, the time complexity of the continuallySplit function is $O(\log n)$, where n represents the input size.



- NOTE: In general, doing something with every item in one dimension is **linear**, doing something with every item in two dimensions is **quadratic**, and dividing the working area in half is **logarithmic**.



Question 1: Calculate the time complexity of the following code snippet

```
int count = 0 ;  
for (int i = n; i > 0; i /= 2)  
    for (int j = 0; j < i; j++)  
        count++;
```

Solution

- Think about how many times **count++** will run.

When $i = N$, it will run N times.

When $i = N / 2$, it will run $N / 2$ times.

When $i = N / 4$, it will run $N / 4$ times.

And so on.

- The total number of times **count++** will run is $N + N/2 + N/4 + \dots + 1 = 2 * N$.
So the time complexity will be $O(N)$.

Question 2: Finding the sum of the first n numbers.

```
int findSum(int n) {  
    return n * (n + 1) / 2;  
}
```

$$S_n = \frac{n}{2} (a + l)$$

Diagram illustrating the formula for the sum of the first n terms of an arithmetic progression:

- S_n : Sum of n terms (indicated by a red arrow)
- n : Number of terms (indicated by a purple arrow)
- a : First term (indicated by a blue arrow)
- l : n th term (indicated by a green arrow)



Solution

- there is only one statement and we know that a statement takes constant time for its execution. The basic idea is that if the statement is taking constant time, then it will take the same amount of time for all the input size and we denote this as $O(1)$.



Question 3: Finding the sum of the first n numbers.

```
int findSum(int n) {  
    int sum = 0;  
    for(int i = 1; i <= n; ++i) {  
        sum += i;  
    }  
    return sum;  
}
```



Solution

Total time taken = time taken by all the statements to execute

here in our example we have 3 constant time taking statements i.e. `"sum = 0"`, `"i = 0"`, and `"return sum"`, so we can add all the constants and replace with some new constant `"c"`

apart from this, we have two statements running n-times i.e. `"i < n(in real n+1)"` and `"sum = sum + i"`
i.e. $c_2 * n + c_3 * n = c_0 * n$

Total time taken = $c_0 * n + c$



- When $n = 1$, the loop will run once.
 - When $n = 2$, the loop will run twice.
 - When $n = 3$, the loop will run three times.
 - And so on, until n , the loop will run n times.
 - In general, the loop will run n times.
-
- So, the time complexity of the findSum function is $O(n)$, where n represents the input size.



- The big O notation of the above code is $O(c_0 * n) + O(c)$, where c and c_0 are constants. So, the overall time complexity can be written as **$O(n)$** .
- In this code, the loop runs from 1 to n (inclusive), and in each iteration, it performs a constant amount of work: incrementing i and adding its value to the sum. The loop runs n times because it starts from 1 and goes up to n .
- The work done in each iteration is constant, and the number of iterations is directly proportional to the value of n .
- Therefore, the time complexity of this code snippet is $O(n)$, where n is the input to the **findSum** function. As n increases, the number of iterations grows linearly, and the time taken by the function also increases linearly.

1. Iterating through Numbers:

- The function iterates from 1 to `n` inclusive.
- This means it performs a constant-time operation for each integer from 1 to `n`.

2. Constant-Time Operation:

- Within the loop, there is a single operation: adding the current value of `i` to the `sum`.
- This operation takes constant time, regardless of the value of `n`.

3. Total Operations:

- Since there's a single operation performed `n` times in the loop, the total number of operations grows linearly with `n`.

4. Time Complexity:

- Because the number of operations is directly proportional to `n`, we say the time complexity of this function is **$O(n)$** , where **O** represents the upper bound or worst-case scenario of time taken relative to the size of the input `n`.

In summary, the time complexity of the `findSum` function is linear, meaning as `n` increases, the time taken by the function also increases linearly. This makes it an efficient algorithm for this particular task.



Question 4: Finding the sum of the first n numbers.

```
int findSum(int n) {  
    int sum = 0;  
    for(int i = 1; i <= n; ++i) {  
        for(int j = 1; j <= i; ++j) {  
            sum++; #NB: this will run [n * (n + 1) / 2]  
        }  
    }  
    return sum;  
}
```



1. When `i = 1`, the inner loop will run once.
2. When `i = 2`, the inner loop will run twice.
3. When `i = 3`, the inner loop will run three times.
4. And so on, until `i = n`, the inner loop will run `i` times.

In general, for each value of `i`, the inner loop will run `i` times.

So, the total number of iterations of the inner loop is the sum of the first n positive integers, which is $\frac{n \cdot (n+1)}{2}$.

Therefore, the time complexity of the `findSum` function is $O(n^2)$, as it has a nested loop structure where each loop iterates up to n .



- Total time taken = time taken by all the statements to execute

the statement that is being executed most of the time is `"sum++"` i.e. $n * (n + 1) / 2$

So, total complexity will be: $c1*n^2 + c2*n + c3$ [$c1$ is for the constant terms of n^2 , $c2$ is for the constant terms of n , and $c3$ is for rest of the constant time]



- In this code, you have two nested loops. The outer loop iterates from $i = 1$ to $i \leq n$, and the inner loop iterates from $j = 1$ to $j \leq i$. Inside the inner loop, you're performing a constant-time operation ($\text{sum}++$).
- The number of iterations of the inner loop is determined by the value of i . In the first iteration of the outer loop, the inner loop runs 1 time. In the second iteration of the outer loop, the inner loop runs 2 times. In the third iteration, the inner loop runs 3 times, and so on.
- The total number of iterations of the inner loop across all iterations of the outer loop can be approximated as:
 $1 + 2 + 3 + \dots + n$
- This sum is equal to $n * (n + 1) / 2$, which is proportional to n^2 for large values of n .
- Therefore, the overall time complexity of this code snippet is $O(n^2)$, indicating a quadratic relationship between the input size and the number of operations performed. As n grows larger, the number of operations increases quadratically.

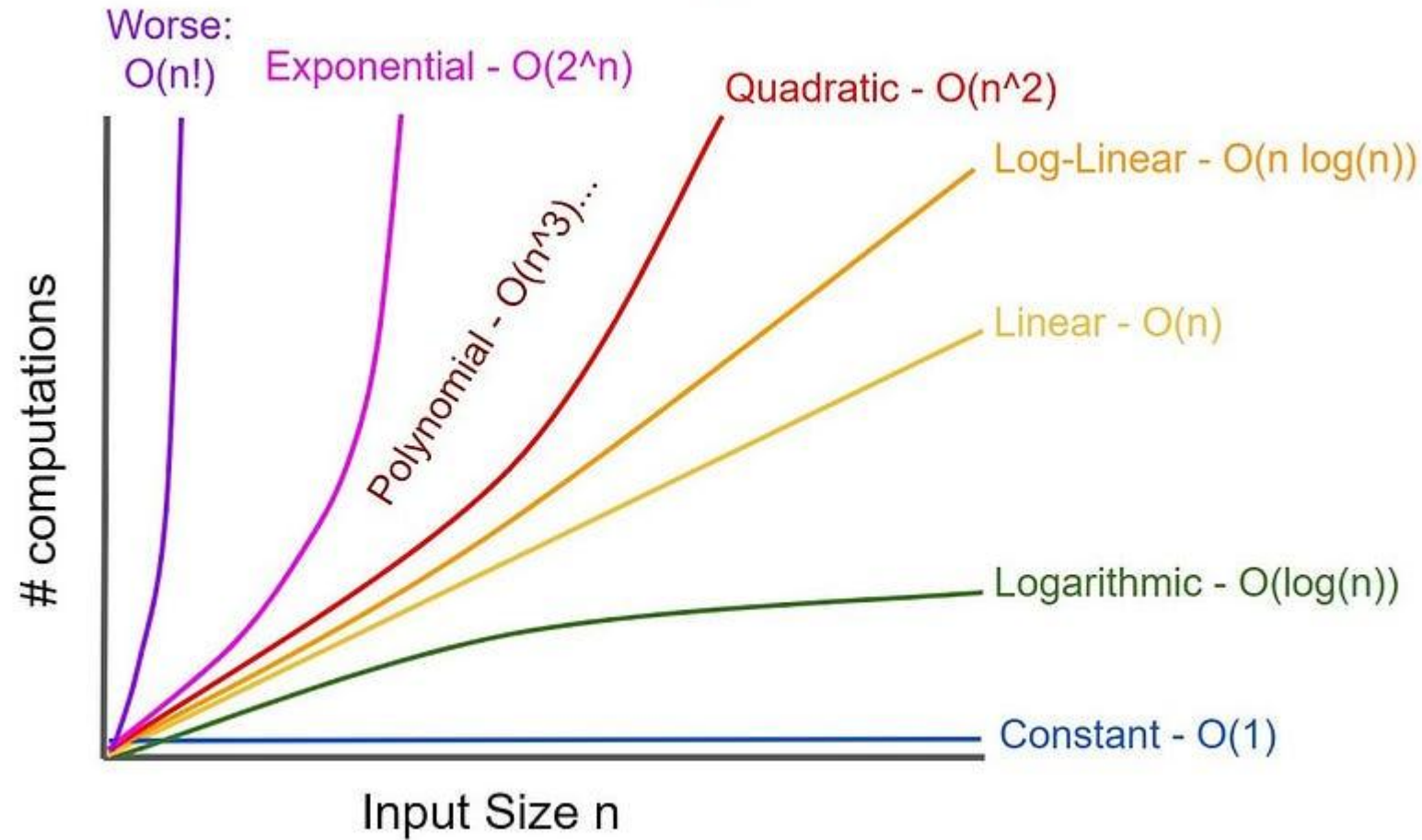
Question 5:

```
int countPlusPlus(int n) {  
    int count = 0;  
    for (int i = 0; i < n; i++)  
        for (int j = 0; j < i; j++)  
            count++;  
    return count;  
}
```

Solution

- Lets see how many times **count++** will run.
- When $i = 0$, it will run 0 times.
When $i = 1$, it will run 1 times.
When $i = 2$, it will run 2 times and so on.
- Total number of times **count++** will run is
 $0+1+2+\dots+(N-1) = N*(N-1) / 2$
So the time complexity will be $O(n^2)$.

Time Complexities Graph





Group Assignment

- Write an essay on Space Complexity. Back your explanations with examples.

